

Module : Business Intelligence

Filière : 2ACI INF grp A

RAPPORT DE TP N°3

Intégration de Données (ETL) avec Pentaho

Implémentation et Automatisation du Data Warehouse hôtelier

Réalisé par :
Youssef FELLAH

Encadré par :
Pr. Abdelmajid BOUSSELHAM

Année universitaire : 2025/2026

1 Introduction et Environnement

L'objectif de ce TP est d'automatiser le chargement du Data Warehouse conçu précédemment en utilisant **Pentaho Data Integration (Spoon)**. L'environnement technique repose sur un système **Fedora 43** hébergeant une base de données **PostgreSQL** active via Docker.

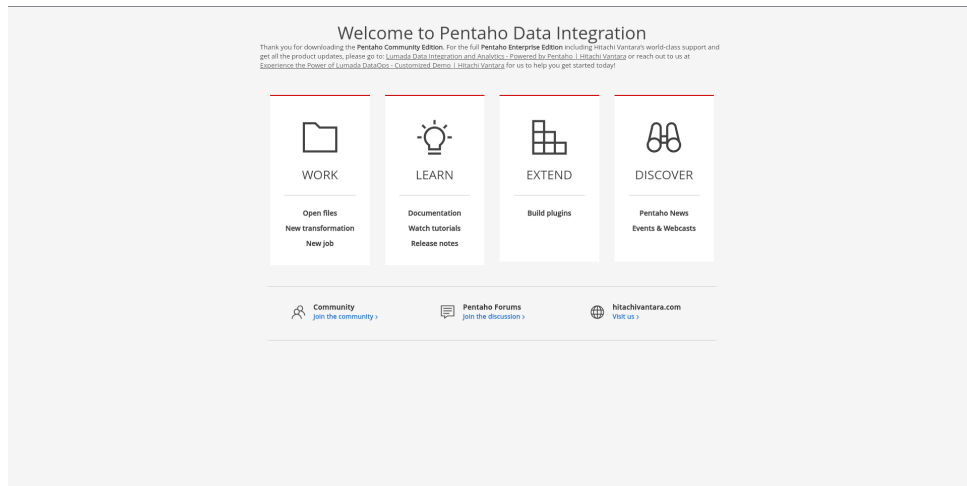


FIGURE 1 – Interface d'accueil de Pentaho Data Integration (Spoon)

2 Développement des Transformations (ETL)

2.1 Chargement de la Dimension Client (dim_client)

Le flux débute par l'extraction des données brutes depuis le fichier `clients.csv`. Nous avons configuré le délimiteur (virgule) et validé la lecture des colonnes via l'aperçu des données.

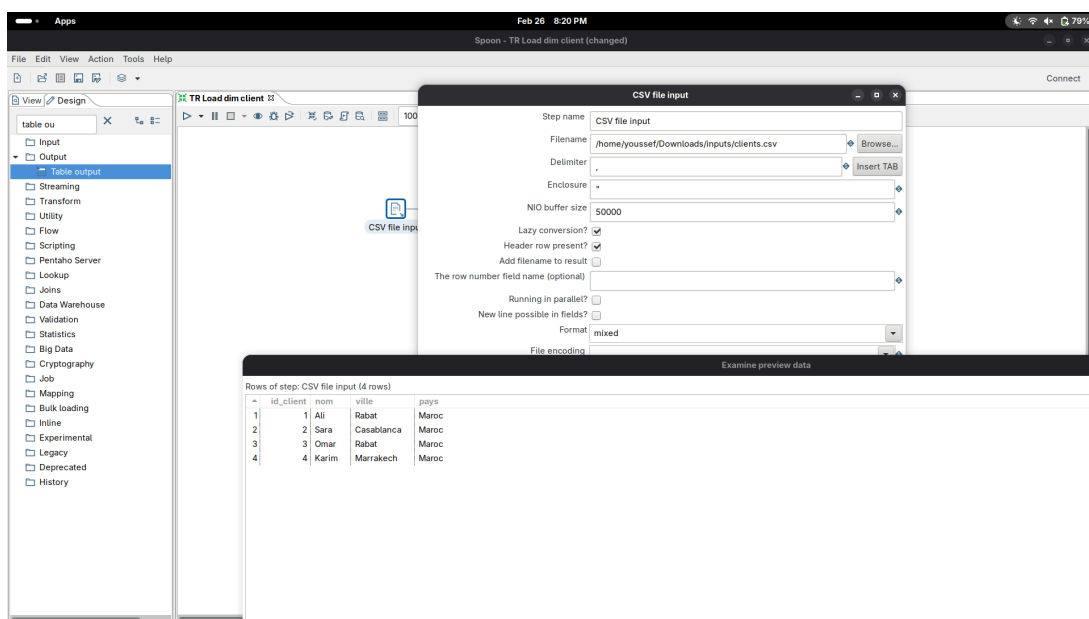


FIGURE 2 – Configuration de l'étape CSV File Input et aperçu des clients

Pour assurer la cohérence du schéma, l'étape **Select Values** a été ajoutée pour définir les métadonnées : conversion du champ `id_client` en type **Integer** et nom en **String**.

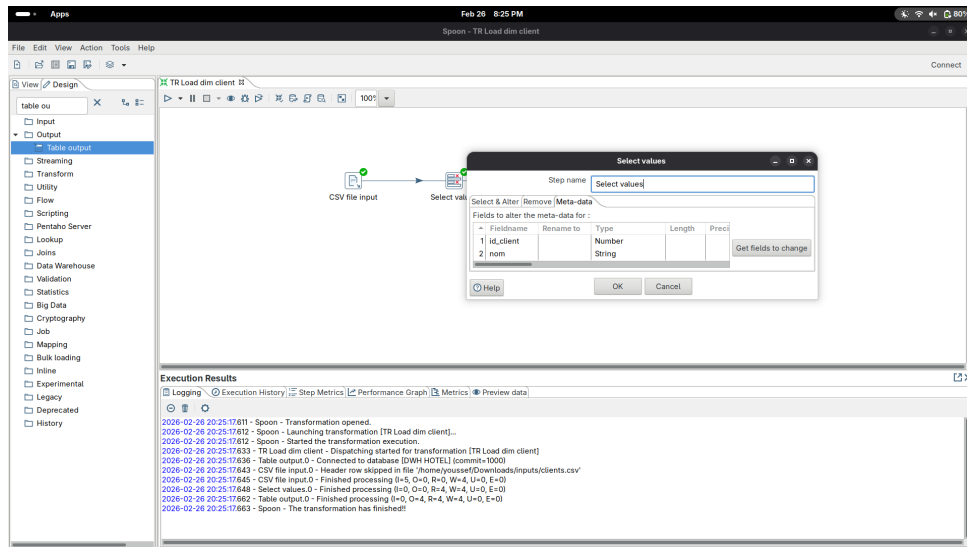


FIGURE 3 – Ajustement des métadonnées pour la dimension client

Enfin, les données sont injectées dans la table cible `dim_client` de PostgreSQL en utilisant la connexion **DWH HOTEL** préalablement configurée.

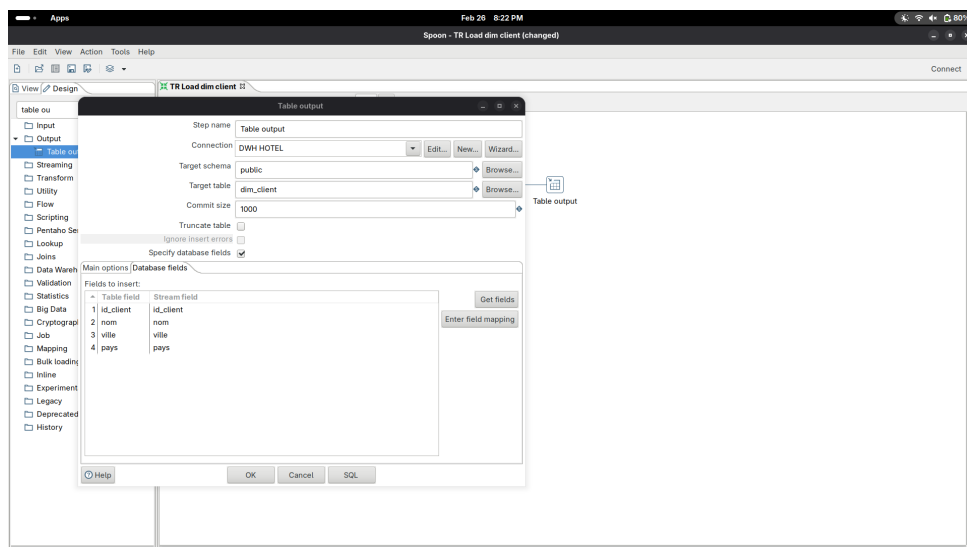
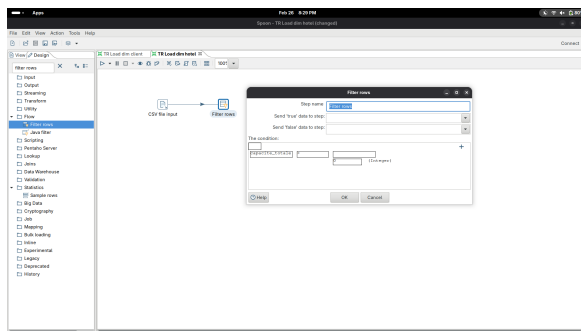


FIGURE 4 – Configuration de la table de sortie `dim_client`

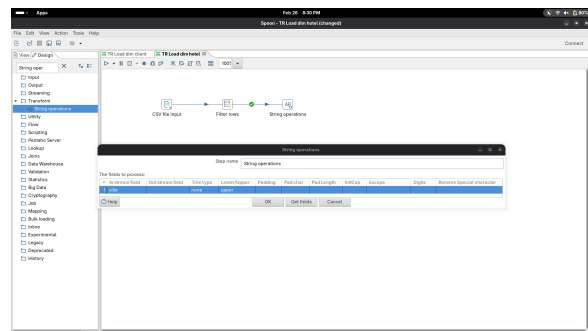
2.2 Dimension Hôtel avec Nettoyage et Normalisation

Le chargement de la dimension hôtel intègre des étapes de nettoyage essentielles :

- **Filtrage** : Utilisation d'un step *Filter Rows* pour ne conserver que les lignes où la `capacite_totale` est strictement positive (> 0).
- **Normalisation** : Application d'un step *String Operations* sur le champ `ville` pour uniformiser les données en majuscules (Uppercase).



(a) Condition de filtre : Capacité > 0



(b) Passage des villes en majuscules

FIGURE 5 – Opérations de nettoyage et normalisation pour dim_hotel

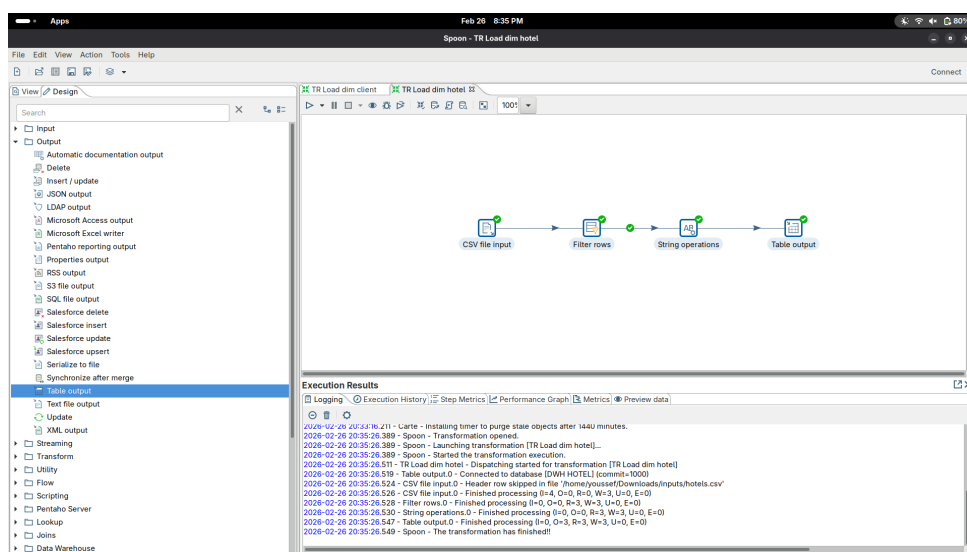


FIGURE 6 – Flux complet de chargement de dim_hotel avec succès

2.3 Dimension Temps et Calculs Analytiques

Pour la dimension temps, nous avons transformé la date brute du fichier `reservations.csv`. Le step **Unique Rows** a été utilisé sur `date_reservation` pour éviter toute redondance.

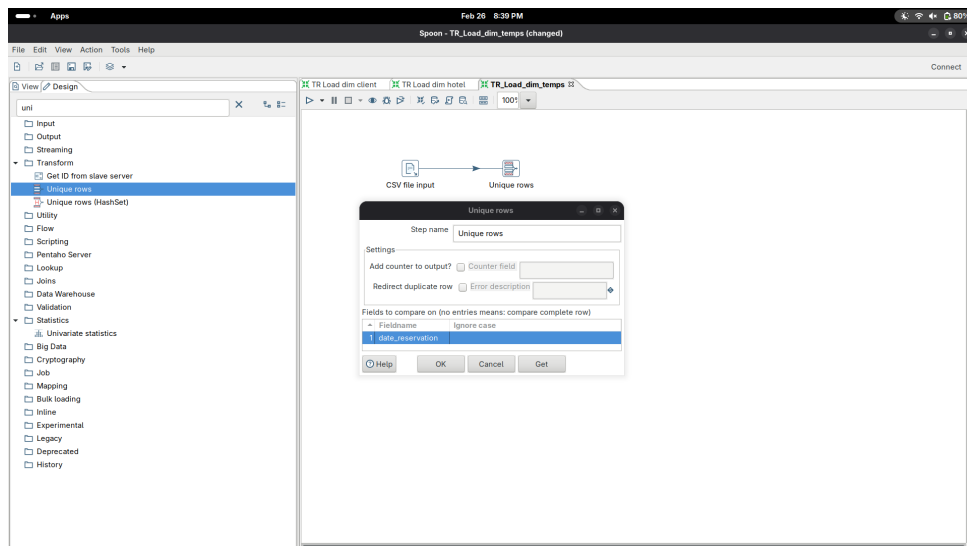


FIGURE 7 – Étape de suppression des doublons sur la date

L'enrichissement des données a été réalisé avec le step **Calculator**, permettant d'extraire automatiquement le jour, le mois, l'année et de calculer le trimestre analytique.

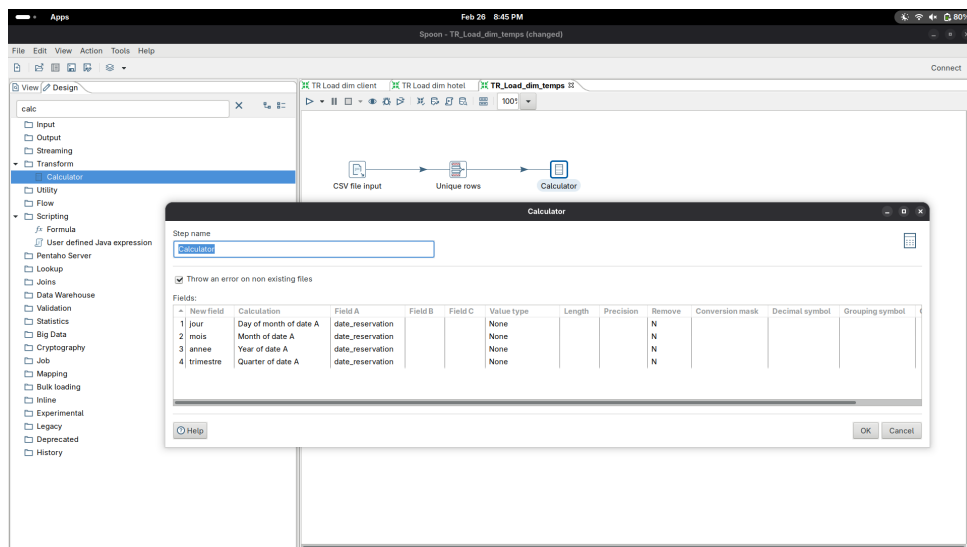
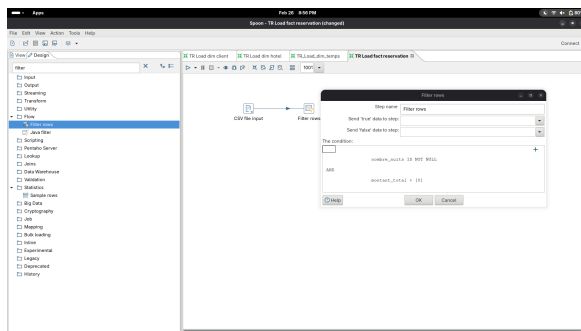


FIGURE 8 – Extraction des composants temporels via Calculator

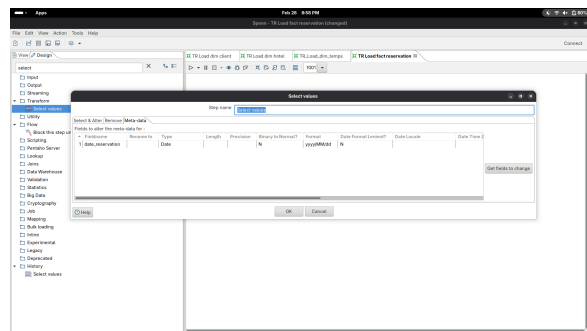
3 Table de Faits et Orchestration

3.1 Alimentation de fact_reservation

Le chargement de la table de faits (fact_reservation) applique des règles de gestion strictes : filtrage des montants nuls et gestion des valeurs manquantes pour le nombre de nuits. Le format de date a été normalisé en *yyyy-MM-dd* dans les métadonnées.



(a) Filtrage des faits (nuits et montants)



(b) Formatage de la date de réservation

FIGURE 9 – Règles de nettoyage et formatage pour la table de faits

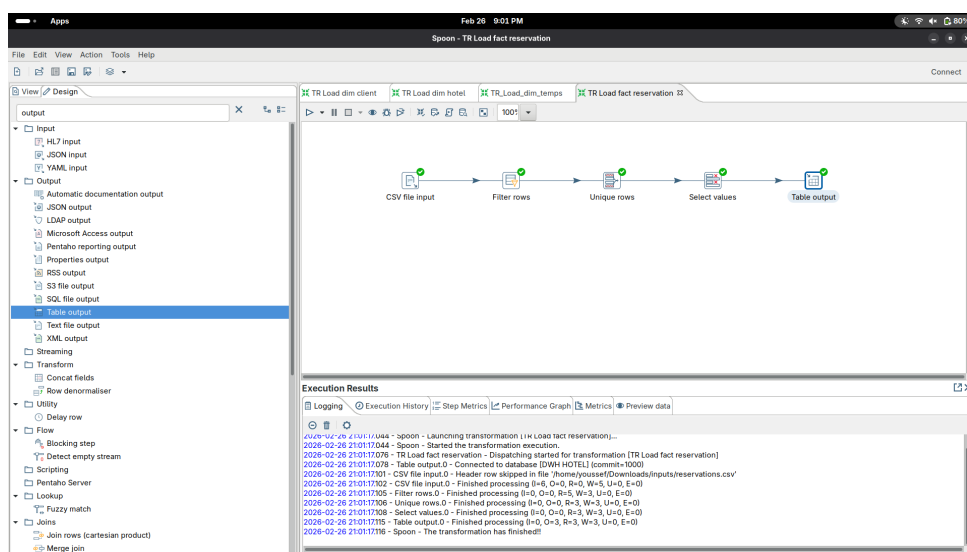


FIGURE 10 – Schéma de la transformation fact_reservation validée

3.2 Orchestration Finale par le Job

Afin d'automatiser l'intégralité du processus, un **Job** (.kjb) a été créé. Il définit l'ordonancement séquentiel des tâches : chargement des dimensions (clients, hôtels, temps) suivi de la table de faits, garantissant le respect des contraintes d'intégrité référentielle.

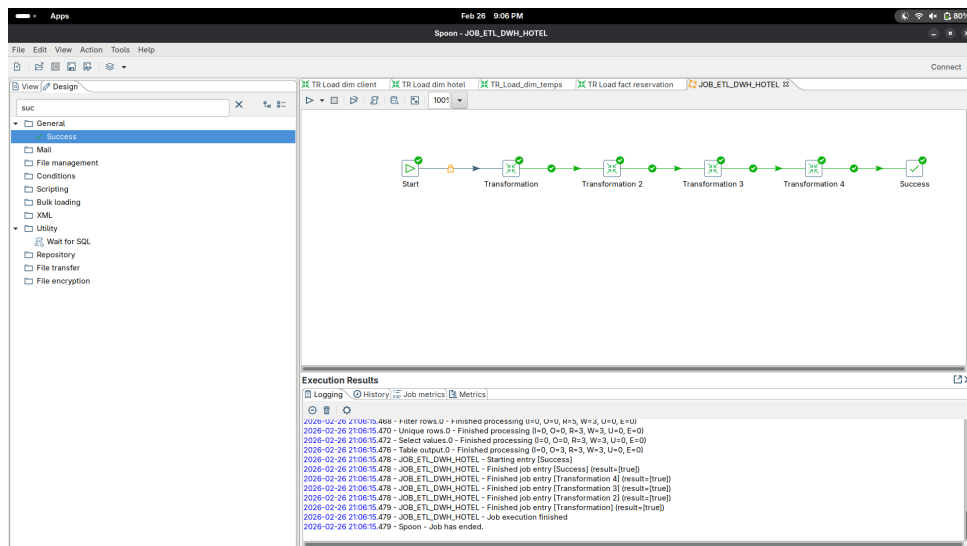


FIGURE 11 – Exécution complète et réussie du Job ETL final

4 Validation Finale

La validation a été effectuée en exécutant des requêtes SQL de comptage (COUNT) directement dans le terminal Docker. Le résultat confirme la présence effective de 3 enregistrements dans la table de faits, correspondant aux données valides traitées.

```

youssef@fedora:~$ docker exec -it relaxed_villani postgresql -U admin -d dwh_hotel
youssef@fedora:~$ docker exec -it relaxed_villani postgresql -U admin -d dwh_hotel
dwh_hotel=# SELECT COUNT(*) FROM fact_reservation;
count
-----
3
(1 row)
dwh_hotel=#

```

FIGURE 12 – Requête de vérification SQL dans PostgreSQL

5 Conclusion

Ce TP a permis de maîtriser l'automatisation des flux ETL complexes. Grâce à Pentaho Spoon, nous avons réussi à extraire, transformer et charger des données hôtelières tout en assurant leur qualité et leur intégrité dans un environnement PostgreSQL.